

GUÍA DOCENTE — CLASE 04

DASHBOARD SAAS COMPLETO

Docente:

Ing. M.Sc. Rudy Luis Manzaneda Veizaga

OBJETIVO DE LA CLASE

Construir un dashboard SaaS completo utilizando Flask, Blueprints, PostgreSQL, SQLAlchemy ORM, Login seguro, importación CSV, APIs JSON, filtros dinámicos, Chart.js y DataTables.

La clase se basa directamente en el proyecto:

`clase4_dashboard_saas_completo`

CONTENIDO REAL DE LA CLASE

La clase integra:

- Flask + Blueprints.
 - PostgreSQL + SQLAlchemy ORM.
 - Login seguro.
 - Importación CSV.
 - APIs para dashboards.
 - KPIs empresariales.
 - Filtros dinámicos.
 - Chart.js.
 - DataTables.
 - Dashboard Ejecutivo.
 - Dashboard Clientes.
 - Dashboard Productos / OLAP.
-

INICIO DE LA CLASE

EXPLICACIÓN DOCENTE

"Buenos días estudiantes.

Hoy trabajaremos el proyecto más completo del módulo.

En esta clase integraremos prácticamente todo lo trabajado anteriormente:

- Flask,
- PostgreSQL,
- ORM,
- login,
- dashboards,
- APIs,
- Chart.js,
- DataTables,
- filtros dinámicos.

El objetivo es construir un dashboard SaaS empresarial completamente funcional."

PARTE 1 — PRESENTACIÓN DEL PROYECTO

EXPLICACIÓN DOCENTE

"Primero observaremos la estructura real del proyecto.

Este proyecto ya no es una aplicación pequeña.

Ahora trabajamos una arquitectura más profesional utilizando Blueprints."

MOSTRAR ESTRUCTURA

```
clase4_dashboard_saas_completo/  
|  
├─ run.py  
├─ requirements.txt  
├─ dataset_cafe_andino.csv  
|
```

```
├─ app/
│   ├── __init__.py
│   ├── config.py
│   ├── models.py
│   ├── forms.py
│   │
│   ├── auth/
│   │   └─ routes.py
│   │
│   ├── dashboard/
│   │   └─ routes.py
│   │
│   ├── data/
│   │   ├── routes.py
│   │   └─ services.py
│   │
│   ├── templates/
│   │   ├── login.html
│   │   ├── dashboard1.html
│   │   ├── dashboard2.html
│   │   ├── dashboard3.html
│   │   └─ import_csv.html
│   │
│   └─ static/
│       ├── css/
│       └─ js/
```

EXPLICACIÓN DOCENTE

“Ahora ya tenemos una arquitectura modular.

Cada módulo tiene una responsabilidad específica.

Por ejemplo:

- auth maneja autenticación.
- dashboard maneja paneles.
- data maneja APIs y servicios.

Esto es muy parecido a proyectos empresariales reales.”

PARTE 2 — LOGIN Y AUTENTICACIÓN

EXPLICACIÓN DOCENTE

“Antes de ingresar al dashboard, el usuario debe autenticarse.

Esto permite controlar sesiones y seguridad.”

MOSTRAR FLUJO

```
Usuario
  ↓
Formulario Login
  ↓
Flask-WTF
  ↓
SQLAlchemy ORM
  ↓
PostgreSQL
  ↓
Validación
  ↓
Dashboard
```

EXPLICACIÓN DOCENTE

“Flask consulta PostgreSQL mediante ORM.

No usamos SQL manual.

El usuario autenticado puede acceder a los dashboards.”

PARTE 3 — IMPORTACIÓN CSV

EXPLICACIÓN DOCENTE

“Ahora veremos cómo importar el dataset Café Andino.”

MOSTRAR COMANDO

```
flask import-csv --path dataset_cafe_andino.csv
```

EXPLICACIÓN DOCENTE

“Este comando carga el CSV hacia PostgreSQL.

El dataset contiene:

- ventas,
- clientes,
- productos,
- categorías,
- regiones,
- fechas,
- costos,
- ganancias.

Posteriormente los dashboards consumen esa información.”

PARTE 4 — DASHBOARDS

EXPLICACIÓN DOCENTE

“Este proyecto tiene 3 dashboards principales.”

DASHBOARD 1 — EJECUTIVO

Explicación

“Este dashboard muestra KPIs generales del negocio.”

KPIs

- Ventas Totales.
- Utilidad Neta.

- Pedidos.
 - Clientes.
 - Margen de Utilidad.
-

DASHBOARD 2 — CLIENTES

Explicación

“Este dashboard analiza comportamiento de clientes.”

Visualizaciones

- Top clientes.
 - Segmentación.
 - Compras por región.
 - Ticket promedio.
-

DASHBOARD 3 — PRODUCTOS / OLAP

Explicación

“Este dashboard analiza productos y agregaciones OLAP.”

Visualizaciones

- Ventas por categoría.
 - Top productos.
 - Ganancias por región.
 - Evolución mensual.
-

PARTE 5 — APIs JSON

EXPLICACIÓN DOCENTE

“Ahora veremos cómo Flask entrega información al frontend.”

MOSTRAR ENDPOINTS

```
/api/dashboard/kpis  
/api/dashboard/ventas-mensuales  
/api/dashboard/top-productos  
/api/dashboard/tabla-ventas
```

EXPLICACIÓN DOCENTE

“Estos endpoints devuelven JSON.

JavaScript consume esa información y actualiza la interfaz.”

PARTE 6 — FETCH API Y AJAX

EXPLICACIÓN DOCENTE

“Ahora veremos cómo el frontend consume las APIs.”

MOSTRAR FETCH

```
const response = await fetch('/api/dashboard/kpis')  
const data = await response.json()
```

EXPLICACIÓN DOCENTE

“Esto permite:

- actualizar KPIs,
- actualizar gráficos,
- actualizar DataTables,
- aplicar filtros,

sin recargar toda la página.”

PARTE 7 — CHART.JS

EXPLICACIÓN DOCENTE

“Chart.js permite construir gráficos dinámicos.”

GRÁFICOS REALES DEL PROYECTO

- Ventas mensuales.
 - Ventas por categoría.
 - Ganancias por región.
 - Top productos.
 - Distribución de clientes.
-

EXPLICACIÓN DOCENTE

“Los gráficos consumen JSON dinámicamente.

Cada filtro modifica automáticamente la visualización.”

PARTE 8 — DATATABLES

EXPLICACIÓN DOCENTE

“DataTables convierte tablas normales en tablas profesionales.”

FUNCIONALIDADES

- Búsqueda.
 - Paginación.
 - Ordenamiento.
 - Filtros.
 - Navegación.
-

EXPLICACIÓN DOCENTE

“Esto mejora mucho la experiencia de usuario en dashboards empresariales.”

PARTE 9 — FILTROS DINÁMICOS

EXPLICACIÓN DOCENTE

“Los filtros permiten analizar diferentes dimensiones del negocio.”

FILTROS DEL PROYECTO

- Año.
 - Región.
 - Categoría.
 - Segmento.
-

EXPLICACIÓN DOCENTE

“Cuando el usuario cambia un filtro:

- Flask procesa la solicitud.
- ORM consulta PostgreSQL.
- JavaScript actualiza gráficos.
- DataTables cambia automáticamente.

Todo ocurre dinámicamente.”

PARTE 10 — COMANDOS PRINCIPALES

MOSTRAR COMANDOS

```
pip install -r requirements.txt
copy .env.example .env
flask db upgrade
flask create-admin
```

```
flask import-csv --path dataset_cafe_andino.csv
python run.py
```

EXPLICACIÓN DOCENTE

“Estos comandos permiten:

- instalar dependencias,
 - configurar variables,
 - crear tablas,
 - crear administrador,
 - importar CSV,
 - ejecutar la aplicación.”
-

PARTE 11 — DEMOSTRACIÓN FINAL

EXPLICACIÓN DOCENTE

“Ahora ejecutaremos el proyecto completo.”

DEMOSTRACIÓN

1. Login.
 2. Dashboard Ejecutivo.
 3. Dashboard Clientes.
 4. Dashboard OLAP.
 5. Cambio de filtros.
 6. Actualización dinámica.
 7. APIs JSON.
 8. Chart.js.
 9. DataTables.
-

PREGUNTAS DOCENTES

Pregunta 1

“¿Qué ventaja tiene usar Blueprints?”

Pregunta 2

“¿Qué devuelve Flask al frontend?”

Pregunta 3

“¿Por qué usamos Fetch API?”

Pregunta 4

“¿Quién consulta PostgreSQL?”

Pregunta 5

“¿Qué ventaja tiene Chart.js?”

Pregunta 6

“¿Por qué DataTables mejora la UX?”

CIERRE DE CLASE

EXPLICACIÓN FINAL

“Hoy construimos una arquitectura SaaS completa.

Ya no trabajamos solamente HTML o Flask básico.

Ahora integramos:

- Flask,
- PostgreSQL,
- ORM,
- Login,
- APIs,
- JSON,
- Fetch,

- Chart.js,
- DataTables,
- Dashboards ejecutivos.

Este tipo de arquitectura es muy cercana a dashboards empresariales reales.”

TAREA PROPUESTA

ACTIVIDAD

Implementar:

1. Un nuevo KPI.
 2. Un gráfico adicional.
 3. Un filtro nuevo.
 4. Un endpoint API.
 5. Una mejora visual responsive.
-

RESULTADO ESPERADO

Al finalizar la clase el estudiante deberá comprender:

- Arquitectura Flask modular.
- Blueprints.
- Integración frontend-backend.
- APIs JSON.
- Comunicación asíncrona.
- Dashboards ejecutivos.
- Chart.js.
- DataTables.
- Dashboards SaaS modernos.